

Cours 11 — Filtres : une formule appliquée à chaque pixel

Fichiers : ofApp11.h + ofApp11.cpp , image data/pandaroux.jpg **Avant** : cours 10 (lire un pixel), cours 09 (HSB).

Un filtre d'image, c'est une règle du genre « pour chaque pixel, remplace sa couleur par $f(\text{couleur})$ ». Dix règles différentes, et une même boucle qui les applique. Touches 0 à 9 pour changer d'effet, souris pour régler le paramètre.



■ 1. Deux images : la source et le résultat

```
// dans le .h
ofImage source;
ofImage resultat;
```

```
// dans setup()
source.load("pandaroux.jpg");
resultat.allocate(source.getWidth(), source.getHeight(), OF_IMAGE_COLOR);
```

On ne modifie **jamais** la source. On lit dedans, on écrit dans `resultat`. Sinon, changer d'effet partirait d'une image déjà transformée, et appliquer deux fois le négatif ne redonnerait pas l'original. `allocate` réserve une image vide de la bonne taille, en couleur.

■ 2. La double boucle : visiter tous les pixels

```

for (int y = 0; y < source.getHeight(); y++) {
    for (int x = 0; x < source.getWidth(); x++) {
        ofColor c = source.getColor(x, y);
        resultat.setColor(x, y, filtre(c, parametre));
    }
}
resultat.update();

```

Une boucle dans une boucle. La boucle extérieure parcourt les lignes, `y` de 0 à la hauteur. Pour **chaque** ligne, la boucle intérieure parcourt les colonnes, `x` de 0 à la largeur. Le corps intérieur s'exécute donc `largeur × hauteur` fois : 400 000 fois pour cette image. C'est le motif fondamental de tout le traitement d'image, et il reviendra dans chaque cours jusqu'à la fin.

À chaque pixel : lire dans la source, transformer, écrire dans le résultat avec `setColor(x, y, couleur)`. Après la boucle, `resultat.update()` envoie les nouveaux pixels à la carte graphique. Sans lui, `draw()` continue d'afficher l'ancienne version.

Tout ceci est dans `update()`, pas dans `draw()`. `draw()` se contente de `resultat.draw(0, 0)`.

■ 3. Une fonction qui renvoie une valeur

```

ofColor ofApp::filtre(ofColor c, float p) {
    ...
    return ofColor(255 - r, 255 - g, 255 - b);
}

```

Jusqu'ici nos fonctions étaient `void` : elles faisaient quelque chose sans rien rendre. Celle-ci commence par `ofColor` : elle **renvoie** une couleur. Le mot `return` donne la valeur renvoyée et termine la fonction immédiatement. Celui qui appelle récupère cette valeur là où il a écrit `filtre(c, parametre)`, comme `ofRandom` ou `getColor` renvoient un résultat.

Une fonction qui reçoit un pixel et en renvoie un autre, c'est exactement la définition d'un filtre par pixel.

■ 4. Le piège du débordement

```

float r = c.r;
float g = c.g;
float b = c.b;

```

Les composantes d'une `ofColor` sont des `unsigned char` : la plus petite case de la mémoire, **un seul octet** (8 bits, donc $2^8 = 256$ valeurs, de 0 à 255 — `unsigned` : sans signe). La grande case du cours 01 (`float`, 4 octets) serait un luxe : l'écran ne connaît que 256 niveaux par canal, et une `ofColor` tient ainsi en trois octets collés — la `struct` du cours 07 :

```

ofColor c :  +-----+-----+-----+
              | r   | g   | b   |      trois octets collés : c.r, c.g, c.b
              | 250 | 80  | 32  |
              +-----+-----+-----+

```

Mais la petite case déborde vite : au-delà de 255, le compteur **reboucle**, comme un compteur kilométrique qui repasse à zéro.

```
... 253 254 255 -> 0 1 2 ...      250 + 10 = 4, pas 260 !
```

Un filtre « luminosité + 60 » sur un pixel clair donnerait donc des taches noires.

La règle : on **copie dans des float**, on calcule, puis on borne avec `ofClamp(valeur, 0, 255)` avant de reconstruire la couleur :

```
float k = (p - 0.5f) * 256;
return ofColor(ofClamp(r + k, 0, 255), ofClamp(g + k, 0, 255), ofClamp(b + k, 0, 255));
```

■ 5. Choisir l'effet : `switch` et le clavier

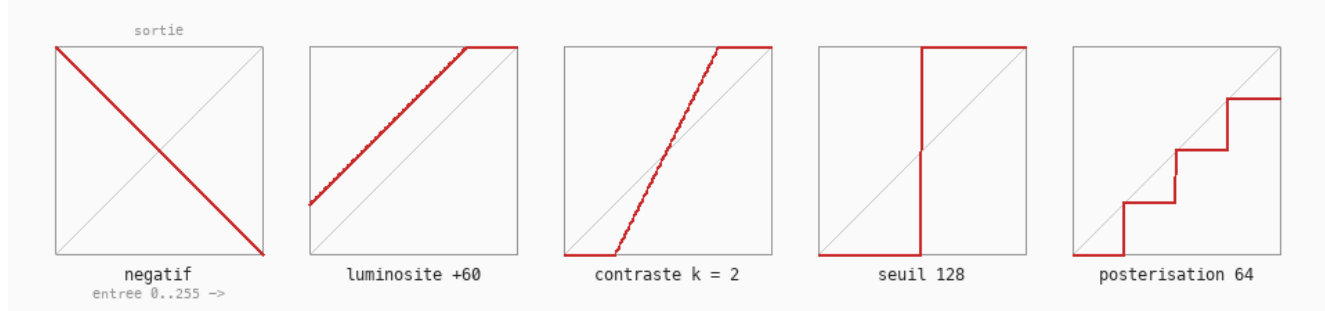
```
switch (effet) {
case 1:
    return ofColor(255 - r, 255 - g, 255 - b);
case 2: {
    float gris = (r + g + b) / 3;
    return ofColor(gris, gris, gris);
}
...
default:
    return c;
}
```

`switch (variable)` compare la variable à chaque `case` et saute au bon. C'est un `if` à plusieurs branches, plus lisible que dix `if` à la suite. `default` est la branche « aucun des cas ». Quand un `case` déclare une variable, il faut des accolades autour de son bloc. Ici chaque branche se termine par `return`, ce qui sort de la fonction ; sinon on écrirait `break` pour sortir du `switch`.

```
void ofApp::keyPressed(int key) {
    if (key >= '0' && key <= '9') {
        effet = key - '0';
    }
}
```

`keyPressed` est un bloc appelé automatiquement à chaque touche pressée. `key` est le **code** du caractère : `'0'` vaut 48, `'1'` 49, etc. `key - '0'` transforme le code en chiffre. Le `&&` se lit « et » : les deux conditions doivent être vraies.

■ 6. Les formules



Chaque filtre est une petite formule. La courbe ci-dessus montre, pour chaque valeur d'entrée, la valeur de sortie.

Effet	Formule	Ce qui se passe
Négatif	<code>255 - c</code>	chaque canal est retourné
Gris moyenne	<code>(r + g + b) / 3</code>	les trois canaux reçoivent la même valeur
Gris luminance	<code>0.299 r + 0.587 g + 0.114 b</code>	l'œil est plus sensible au vert : un vert pur paraît plus clair qu'un bleu pur
Luminosité	<code>c + k</code>	tout monte ou descend d'un même cran
Contraste	<code>(c - 128) * k + 128</code>	on écarte ($k > 1$) ou rapproche ($k < 1$) du gris moyen
Seuil	<code>c < s ? 0 : 255</code>	noir ou blanc, rien entre
Postérisation	<code>floor(c / pas) * pas</code>	arrondi à un multiple : moins de niveaux, des aplats

La postérisation utilise la **division entière** du cours 01, cette fois volontairement : `floor(137 / 32) * 32` vaut 128. Le seuil utilise la luminance comme critère.

■ 7. Deuxième partie : passer par HSB

```
case 8: {
    float teinte = fmod(c.getHue() + p * 255, 255);
    return ofColor::fromHsb(teinte, c.getSaturation(), c.getBrightness());
}
```

Décaler toutes les couleurs sur la roue : lire la teinte, l'augmenter, faire boucler avec `fmod`, reconstruire. Une ligne en HSB, et pratiquement impossible à écrire en RGB. C'est pour ce genre de filtre qu'on a fait le cours 09.

```
case 9: {
    float cible = p * 255;
    float ecart = fabs(c.getHue() - cible);
    if (ecart > 127) ecart = 255 - ecart;
    if (ecart < 20) return c;
    return ofColor(luminance);
}
```

Le **color splash** : on garde la couleur seulement autour d'une teinte cible, tout le reste passe en gris. `fabs` est la valeur absolue pour les `float`. Le `if (ecart > 127)` gère le fait que la roue est circulaire : les teintes 250 et 5 sont voisines. `ofColor(luminance)` avec un seul nombre fabrique un gris.

1. Sépia : $r' = 0.393 r + 0.769 g + 0.189 b$, $g' = 0.349 r + 0.686 g + 0.168 b$, $b' = 0.272 r + 0.534 g + 0.131 b$. Chaque canal de sortie mélange les trois canaux d'entrée.
2. Échange de canaux : `ofColor(g, b, r)`. Puis isoler un canal : `ofColor(r, 0, 0)`.
3. Affiche la teinte seule en niveaux de gris : `ofColor(c.getHue())`. Puis la saturation, puis la luminosité. Tu **vois** les trois axes du cours 09.
4. Gamma : $255 * \text{pow}(r / 255, k)$ sur chaque canal, avec k de 0.3 à 3 selon la souris.
5. Chromakey : si la teinte est proche du vert, remplace le pixel par celui d'une deuxième image.

■ Le code complet, pas à pas

Le programme entier, dans l'ordre des fichiers. Les blocs ci-dessous mis bout à bout donnent exactement `ofApp11.cpp`.

Le fichier `ofApp11.h`

La table des matières du programme : les blocs qui existent et les variables partagées entre eux, avec leurs valeurs de départ.

```
#pragma once
#include "ofMain.h"

// 11 - Filtrés : une formule appliquée à chaque pixel
// Nouveau : pas de sketch Processing d'origine
// Notions : double boucle sur tous les pixels (lignes / colonnes), image source et image résultat,
//           setColor + update(), débordement des unsigned char (calcul en float puis ofClamp),
//           paramètre à la souris, choix de l'effet au clavier (keyPressed, switch),
//           deuxième partie : les filtres qui passent par HSB (relit 09)
// Ressource : bin/data/pandaroux.jpg (774 x 516)

class ofApp : public ofBaseApp {
public:
    void setup();
    void update();
    void draw();
    void keyPressed(int key);

    // Un pixel entre, un pixel sort. p est le paramètre (0 à 1) piloté par la souris.
    ofColor filtre(ofColor c, float p);

    ofImage source;
    ofImage resultat;
    int effet = 0;
    float parametre = 0.5f;
    std::vector<std::string> noms = {
        "original", "negatif", "gris (moyenne)", "gris (luminance)", "luminosite",
        "contraste", "seuil", "posterisation", "rotation de teinte", "color splash"
    };
};
```

L'inclusion du `.h`, qui rend visibles les variables partagées et les blocs déclarés.

```
#include "ofApp11.h"
```

Étape 1 — `setup()`

Exécuté une fois au lancement : la fenêtre, les chargements, les valeurs de départ.

```
void ofApp::setup() {
    ofSetWindowShape(774, 516);
    if (!source.load("pandaroux.jpg")) {
        ofLogError() << "pandaroux.jpg introuvable dans bin/data";
    }
    // Deux images : on lit dans la source, on écrit dans le résultat.
    // On ne modifie jamais la source : sinon changer d'effet partirait d'une image déjà transfo
    // allocate réserve une image vide de la même taille, en couleur (3 canaux).
    resultat.allocate(source.getWidth(), source.getHeight(), OF_IMAGE_COLOR);
}
```

Étape 2 — `filtre()`

Fonction `filtre()`.

```

ofColor ofApp::filtre(ofColor c, float p) {
    // Les composantes d'un ofColor sont des unsigned char : entiers de 0 à 255, et 255 + 10
    // ne donne pas 260 mais 4 (ça reboucle). On copie donc dans des float, on calcule,
    // et on borne avec ofClamp avant de reconstruire la couleur.
    float r = c.r;
    float g = c.g;
    float b = c.b;

    // Luminance : gris "perçu". L'oeil est bien plus sensible au vert qu'au bleu,
    // donc un vert pur paraît plus clair qu'un bleu pur de même intensité.
    float luminance = 0.299f * r + 0.587f * g + 0.114f * b;

    switch (effet) {

    case 1: // Négatif : chaque canal est retourné
        return ofColor(255 - r, 255 - g, 255 - b);

    case 2: { // Gris par moyenne : les trois canaux reçoivent la même valeur
        float gris = (r + g + b) / 3;
        return ofColor(gris, gris, gris);
    }

    case 3: // Gris par luminance (un seul argument à ofColor = gris)
        return ofColor(luminance);

    case 4: { // Luminosité : on ajoute la même valeur partout, de -128 à +128 selon la souris
        float k = (p - 0.5f) * 256;
        return ofColor(ofClamp(r + k, 0, 255), ofClamp(g + k, 0, 255), ofClamp(b + k, 0, 255));
    }

    case 5: { // Contraste : on écarte (k > 1) ou on rapproche (k < 1) chaque valeur du milieu 128
        float k = p * 3;
        return ofColor(ofClamp((r - 128) * k + 128, 0, 255),
            ofClamp((g - 128) * k + 128, 0, 255),
            ofClamp((b - 128) * k + 128, 0, 255));
    }

    case 6: // Seuil : noir ou blanc, rien entre les deux. Le seuil suit la souris.
        if (luminance < p * 255) return ofColor(0);
        else return ofColor(255);

    case 7: { // Postérisation : on arrondit chaque canal à un multiple de "pas".
        // Division entière (cours 01) : floor(137 / 32) * 32 = 128. Moins de niveaux = ap
        float pas = 8 + p * 120; // 8 à 128
        return ofColor(floor(r / pas) * pas, floor(g / pas) * pas, floor(b / pas) * pas);
    }

    // ---- Deuxième partie : passer par HSB (cours 09) ----
    // Ces deux effets sont une ligne en HSB et une horreur en RGB.

```

```

        case 8: { // Rotation de teinte : on décale la teinte, saturation et luminosité inchangées
            float teinte = fmod(c.getHue() + p * 255, 255);
            return ofColor::fromHsb(teinte, c.getSaturation(), c.getBrightness());
        }

        case 9: { // Color splash : couleur conservée seulement autour d'une teinte cible, gris ailleurs
            float cible = p * 255;
            float ecart = fabs(c.getHue() - cible);
            if (ecart > 127) ecart = 255 - ecart; // la roue est circulaire : 250 et 5 sont proches
            if (ecart < 20) return c;
            return ofColor(luminance);
        }

        default: // 0 : image d'origine
            return c;
    }
}

```

Étape 3 — `update()`

Exécuté à chaque frame, avant le dessin : tout ce qui change.

```

void ofApp::update() {
    parametre = mouseX / (float)ofGetWidth(); // 0 à gauche, 1 à droite

    // Pour chaque ligne y, pour chaque colonne x : lire, transformer, écrire.
    // Cette double boucle revient en 12, sur une grille de 20 pixels au lieu de chaque pixel.
    for (int y = 0; y < source.getHeight(); y++) {
        for (int x = 0; x < source.getWidth(); x++) {
            ofColor c = source.getColor(x, y);
            resultat.setColor(x, y, filtre(c, parametre));
        }
    }

    // setColor modifie les pixels en mémoire ; update() envoie la nouvelle image à la carte graphique
    resultat.update();

    // 774 x 516 = 400 000 pixels par frame : ça passe pour ces filtres.
    // Si ça rame : ne recalculer que quand la souris ou l'effet a changé (voir 17),
    // ou lire le tableau de pixels directement au lieu de getColor / setColor.
}

```

Étape 4 — `draw()`

Exécuté à chaque frame, après `update()` : uniquement du dessin.


```

void ofApp::draw() {
    ofSetColor(255);
    resultat.draw(0, 0);

    ofSetColor(255, 200, 0);
    ofDrawBitmapString("Effet " + ofToString(effet) + " : " + noms[effet]
        + " parametre = " + ofToString(parametre, 2), 10, 20);
    ofDrawBitmapString("Touches 0 a 9 : changer d'effet. Souris : parametre.", 10, 40);
}

```

Étape 5 — keyPressed()

Appelé automatiquement à chaque touche pressée.

```

void ofApp::keyPressed(int key) {
    // key est le code du caractère : '0' vaut 48, '1' vaut 49... d'où la soustraction.
    if (key >= '0' && key <= '9') {
        effet = key - '0';
    }
}

```

En fin de fichier

Les notes et pistes laissées en commentaire dans le code.

```

// Exercices :
// - sépia : chaque canal de sortie mélange les trois canaux d'entrée
//   r' = 0.393 r + 0.769 g + 0.189 b, g' = 0.349 r + 0.686 g + 0.168 b, b' = 0.272 r + 0.534 g + 0.1
// - échange de canaux : ofColor(g, b, r). Isoler un canal : ofColor(r, 0, 0)
// - afficher la teinte, la saturation ou la luminosité seule, en niveaux de gris
// - gamma : 255 * pow(r / 255, k), même chose pour g et b
// - chromakey : si la teinte est proche du vert, remplacer le pixel par une autre image

```

■ Ce qu'il faut retenir

- Un filtre par pixel = une fonction `ofColor f(ofColor c)` appliquée à chaque pixel par une double boucle `for y { for x { } }`.
- Lire dans `source`, écrire dans `resultat` avec `setColor`, puis `resultat.update()`.
- Les composantes de couleur ne dépassent pas 255 et rebouclent : calculer en `float`, borner avec `ofClamp`.
- `return` renvoie une valeur et termine la fonction.
- `switch (v) { case 1: ... default: ... }` choisit une branche ; `keyPressed(int key)` reçoit le code de la touche, `key - '0'` donne le chiffre.
- Certains filtres sont triviaux en HSB et infaisables en RGB : rotation de teinte, color splash.